Framework-ul tău de colecții — Exerciții

Ștefan · 7 pași + provocare · fiecare pas = un commit

Proiectul: repo-ul `colections-csharp`, proiectul existent `Colections` (.NET 8). Framework-ul stă în folderul `Colectii/`, pe care îl creezi în proiect: un fișier per tip, fiecare cu `namespace Colectii;`. Nu adaugi alt proiect. Fișele sunt în `etapa-1-framework/`.
`Program.cs`: `using Colectii;` sus; în clasa `Program`, funcțiile `static void PasN()` din fișă, copiate ca atare, iar `Main` le cheamă pe toate cele scrise până acum, în ordine. Ieșirea fiecărui pas trebuie să fie **identică** cu cea din fișă și să rămână așa după fiecare pas nou.
Reguli în `Colectii/`: doar tablouri `T[]` și clasele tale. Fără `List`, `Dictionary`, `HashSet`, `Stack`, `Queue`, fără LINQ. Din `System.Collections.Generic` ai voie doar la `IEnumerable<T>`, `IEnumerator<T>`, `EqualityComparer<T>`, `IEqualityComparer<T>` și `KeyNotFoundException`.

Pas 1 — `IColectie<T>`, `ILista<T>`, `ListaInlantuita<T>`

Pornești de la `Lista.cs` și `Node.cs`, pe care le ai deja. Citește întâi TEORIE §1: forma lor de acum are cinci probleme de mecanism. Lista ta devine `Colectii/ListaInlantuita.cs` și implementează contractul de mai jos.

```
public interface IColectie<T> : IEnumerable<T>
{
    int Count { get; }
    void Adauga(T element);
    bool Sterge(T element);
    bool Contine(T element);
    void Goleste();
    T[] CaTablou();
}

public interface ILista<T> : IColectie<T>
{
    T this[int index] { get; set; }
    void Insereaza(int index, T element);
    void StergeLa(int index);
    int IndexOf(T element);
}
```

Membrul	Ce face
`Count`	câte elemente sunt acum
`Adauga(x)`	pune `x` la final
`Sterge(x)`	scoate **prima** apariție a lui `x`; `true` dacă a găsit-o
`Contine(x)`	`true` dacă `x` e în colecție
`Goleste()`	`Count` devine 0
`CaTablou()`	tablou **nou**, cu exact `Count` elemente, în ordinea de parcurgere
`this[i]`	citește / scrie elementul de pe poziția `i`
`Insereaza(i, x)`	`x` ajunge pe poziția `i`, cele de după se mută la dreapta; `i == Count` e voie (= la final)
`StergeLa(i)`	scoate elementul de pe poziția `i`, cele de după coboară

`IndexOf(x)`	poziția primei apariții, sau `-1`

`public class ListaInlantuita<T> : ILista<T>`:

1. `private class Nod`, **în interiorul** listei, cu `T Valoare` și `Nod? Urmator`, amândouă modificabile;
2. câmpurile `Nod? cap` (primul nod), `Nod? coada` (ultimul nod) și `int count`; constructorul fără parametri face o listă goală;
3. metodele primesc și întorc `T`, niciodată un nod;
4. `Adauga` leagă nodul nou după `coada`, fără să parcurgă lista;
5. `this[i]`, `Insereaza`, `StergeLa` merg din nod în nod, de la `cap`;
6. `GetEnumerator()` cu `yield return`, de la `cap` până la `null`;
7. `ToString()` întoarce elementele între paranteze drepte, separate de `", "`: `[a, b, c]`.

Funcția `DemoLista` primește **doar** interfața. Nu știe ce listă are în mână.

```
static void DemoLista(ILista<string> lista)
{
    Console.WriteLine(lista + " Count=" + lista.Count);
    lista.Adauga("b");
    lista.Adauga("d");
    lista.Insereaza(0, "a");
    lista.Insereaza(2, "c");
    lista.Insereaza(4, "e");
    Console.WriteLine(lista + " Count=" + lista.Count);
    lista.StergeLa(0);
    lista.StergeLa(3);
    Console.WriteLine(lista.Sterge("c") + " " + lista.Sterge("x"));
    lista.Adauga("f");
    lista[1] = "D";
    Console.WriteLine(lista + " Count=" + lista.Count);
    Console.WriteLine(lista.IndexOf("f") + " " + lista.IndexOf("a"));
    Console.WriteLine(lista.Contine("b") + " " + lista.Contine("a"));
    foreach (string s in lista)
    {
        Console.Write(s + ";");
    }
    Console.WriteLine();
    Console.WriteLine(string.Join("|", lista.CaTablou()));
    lista.Goleste();
    lista.Adauga("z");
    Console.WriteLine(lista + " " + lista[0] + " Count=" + lista.Count);
}

static void Pas1()
{
    DemoLista(new ListaInlantuita<string>());
}
```

Ieșire așteptată:

```
[] Count=0
[a, b, c, d, e] Count=5
True False
[b, D, f] Count=3
2 -1
True False
b;D;f;
b|D|f
[z] z Count=1
```

Teorie: §1, §2, §6

Indiciu: două capcane de compilare. `==` nu merge între două valori de tip `T`: compari cu `EqualityComparer<T>.Default.Equals(a, b)`. Și `IEnumerable<T>` cere **două** `GetEnumerator`: al doilea, ne-generic, îl scrii explicit (`IEnumerator IEnumerable.GetEnumerator()`, cu `using System.Collections;`) și cheamă primul. Dacă după `StergeLa(3)` și `Adauga("f")` litera `f` lipsește, întreabă-te unde mai arată `coada` după ce ai șters ultimul nod.

Gata când: ieșirea e identică, buildul are 0 warning-uri, iar în proiect nu mai există nicio clasă de nod publică.

Pas 2 — `ListaTablou<T>`

`public class ListaTablou<T> : ILista<T>`, același contract, alt mecanism:

1. câmpurile `private T[] elemente` (pornește cu **4** locuri) și `private int count`;
2. `public int Capacitate`: lungimea tabloului din spate (doar `get`);
3. când `Adauga`/`Insereaza` găsește tabloul plin, alocă unul de **2 ori** mai mare și copiază elementele în el;
4. `GetEnumerator()` cu `yield return`, de la poziția 0 la `count - 1`; `ToString()` ca la lista înlănțuită.

```
static void Pas2()
{
    ListaTablou<int> numere = new ListaTablou<int>();
    Console.WriteLine($"Count={numere.Count} " +
                      $"Capacitate={numere.Capacitate}");
    for (int i = 1; i <= 9; i++)
    {
        numere.Adauga(i * 10);
        Console.WriteLine($"+{i * 10}: Count={numere.Count} " +
                          $"Capacitate={numere.Capacitate}");
    }
    Console.WriteLine("---");
    DemoLista(new ListaTablou<string>());
}
```

```
Count=0 Capacitate=4
+10: Count=1 Capacitate=4
+20: Count=2 Capacitate=4
+30: Count=3 Capacitate=4
+40: Count=4 Capacitate=4
+50: Count=5 Capacitate=8
+60: Count=6 Capacitate=8
+70: Count=7 Capacitate=8
+80: Count=8 Capacitate=8
+90: Count=9 Capacitate=16
---
[] Count=0
[a, b, c, d, e] Count=5
True False
[b, D, f] Count=3
2 -1
True False
b;D;f;
b|D|f
[z] z Count=1
```

Teorie: §3

Gata când: ce e după `---` e identic, rând cu rând, cu ieșirea de la Pas 1: același `DemoLista`, două implementări. Iar `Capacitate` sare 4 → 8 → 16 exact la al 5-lea și al 9-lea `Adauga`.

Pas 3 — `ColectieAbstracta<T>`

Ai acum două liste care au scris de două ori aceleași lucruri. `public abstract class ColectieAbstracta<T> : IColectie<T>`, în `Colectii/ColectieAbstracta.cs`:

1. `abstract`: `Count`, `Adauga`, `Sterge`, `Goleste` și `GetEnumerator()` (cel generic);
2. scrise **o singură dată**, aici, doar cu `foreach (T x in this)`: `Contine`, `CaTablou`, `ToString` și `GetEnumerator`-ul ne-generic;
3. `Contine` e `virtual`: o colecție care știe să caute mai repede îl poate suprascrie (pasul 5);
4. ambele liste devin `: ColectieAbstracta<T>, ILista<T>` și din ele dispar cei patru membri de la punctul 2.

```
static void Pas3()
{
    IColectie<int>[] colectii =
        { new ListaInlantuita<int>(), new ListaTablou<int>() };
    foreach (IColectie<int> patrate in colectii)
    {
        for (int i = 1; i <= 5; i++)
        {
            patrate.Adauga(i * i);
        }
        Console.WriteLine(patrate);
        Console.WriteLine(patrate.Contine(16) + " " + patrate.Contine(15));
        int[] tablou = patrate.CaTablou();
        Console.WriteLine(tablou.Length + " " + tablou[4]);
    }
}
```

```
[1, 4, 9, 16, 25]
True False
5 25
[1, 4, 9, 16, 25]
True False
5 25
```

Teorie: §2 (Template Method)

Gata când: ieșirile de la Pas 1 și Pas 2 sunt neschimbate și niciuna dintre liste nu mai conține `Contine`, `CaTablou`, `ToString` sau `GetEnumerator`-ul ne-generic.

Pas 4 — `StivaTablou<T>`, `CoadaCirculara<T>`

```
public interface IStiva<T> : IColectie<T>
{
    void Pune(T element);
    T Scoate();
    T Varf();
}

public interface ICoada<T> : IColectie<T>
{
    void Intra(T element);
    T Iese();
    T Primul();
}
```

`StivaTablou<T> : ColectieAbstracta<T>, IStiva<T>`:

1. tablou de 4 + `count`, dublare când e plin; vârful = ultimul element pus;

2. `Scoate` întoarce vârful și îl scoate; `Varf` doar îl întoarce;
3. `Adauga(x)` = `Pune(x)`; `foreach` merge de la vârf spre bază; `Sterge(x)` scoate prima apariție în ordinea asta.

`CoadaCircularа<T> : ColectieAbstracta<T>, ICoada<T>`:

1. tablou de 4, plus `int primul` (poziția din tablou a primului element) și `int count`;
2. elementul al `i`-lea din coadă stă pe poziția `(primul + i) % elemente.Length`;
3. `Intra` scrie după ultimul; `Iese` întoarce primul element și mută doar `primul` cu o poziție; `Primul()` doar îl întoarce;
4. plin → tablou de 2 ori mai mare, elementele copiate în ordinea din coadă de la poziția 0, iar `primul` devine 0; `public int Capacitate` ca la listă;
5. `Adauga(x)` = `Intra(x)`; `foreach` de la primul la ultimul; `Sterge(x)` scoate prima apariție, iar cele de după ea avansează cu o poziție.

```
static void Pas4()
{
    StivaTablou<char> stiva = new StivaTablou<char>();
    foreach (char c in "abcde")
    {
        stiva.Pune(c);
    }
    Console.WriteLine(stiva + " Varf=" + stiva.Varf());
    Console.WriteLine(stiva.Scoate() + " " + stiva.Scoate() + " " + stiva);

    CoadaCirculara<int> coada = new CoadaCirculara<int>();
    for (int i = 1; i <= 4; i++)
    {
        coada.Intra(i * 10);
    }
    Console.WriteLine(coada + " Capacitate=" + coada.Capacitate);
    Console.WriteLine(coada.Iese() + " " + coada.Iese());
    coada.Intra(50);
    coada.Intra(60);
    Console.WriteLine(coada + " Capacitate=" + coada.Capacitate);
    coada.Sterge(30);
    Console.WriteLine(coada + " Primul=" + coada.Primul());
    coada.Intra(70);
    coada.Intra(80);
    Console.WriteLine(coada + " Capacitate=" + coada.Capacitate);
}
```

```
[e, d, c, b, a] Varf=e
e d [c, b, a]
[10, 20, 30, 40] Capacitate=4
10 20
[30, 40, 50, 60] Capacitate=4
[40, 50, 60] Primul=40
[40, 50, 60, 70, 80] Capacitate=8
```

Teorie: §4

> **Capcana:** coada „naivă”, în care `Iese` mută tot tabloul cu o poziție la stânga, dă aceeași ieșire. Doar că fiecare `Iese` costă cât toată coada.

Gata când: ieșirea e identică, `Iese` nu are nicio buclă, iar pe rândul 5 `Capacitate` e încă 4: 50 și 60 au intrat pe locurile eliberate de 10 și 20.

Pas 5 — `DictionarHash<K, V>`, apoi `MultimeHash<T>`

```
public interface IDictionar<K, V> where K : notnull
{
    int Count { get; }
    V this[K cheie] { get; set; }
    void Adauga(K cheie, V valoare);
    bool IncearcaSaIa(K cheie, [MaybeNullWhen(false)] out V valoare);
    bool ContineCheia(K cheie);
    bool Sterge(K cheie);
    void Goleste();
    IEnumerable<K> Chei { get; }
    IEnumerable<V> Valori { get; }
}
```

`[MaybeNullWhen(false)]` vine din `using System.Diagnostics.CodeAnalysis;` : spune compilatorului că `valoare` poate fi `null` când metoda întoarce `false`.

`DictionarHash<K, V> : IDictionar<K, V> where K : notnull`:

1. clasa internă `Nod` cu `Cheie`, `Valoare`, `Nod? Urmator`; câmpul `Nod?[] galeti` pornește cu **8** găleți; fiecare găleată e începutul unui lanț de noduri;
2. `comparator = EqualityComparer<K>.Default`; găleata unei chei = `(comparator.GetHashCode(cheie) & 0x7FFFFFFF) % galeti.Length`; în lanț compari cu `comparator.Equals`;
3. `public int NumarGaleti` = `galeti.Length`;
4. înainte să pui o cheie **nouă**: dacă `Count + 1 > 0.75 × NumarGaleti`, dublezi numărul de găleți și muți fiecare nod în găleata lui nouă;
5. `this[k]` : citirea unei chei care lipsește aruncă (pasul 6), scrierea înlocuiește sau adaugă; `Adauga` pe o cheie existentă aruncă (pasul 6); `IncearcaSaIa` nu aruncă niciodată;
6. `Chei` și `Valori` cu `yield return`, găleată după găleată.

Ordinea din `Chei` nu e garantată, deci ieșirea trece prin `Sortat`, dată:

```
static string Sortat<T>(IEnumerable<T> elemente)
{
    ListaTablou<T> lista = new ListaTablou<T>();
    foreach (T e in elemente)
    {
        lista.Adauga(e);
    }
    T[] tablou = lista.CaTablou();
    Array.Sort(tablou);
    return string.Join(", ", tablou);
}

static void Pas5a()
{
    DictionarHash<string, int> varste = new DictionarHash<string, int>();
    string[] nume =
        { "Ana", "Ion", "Maria", "Dan", "Ioana", "Vlad", "Elena" };
    for (int i = 0; i < nume.Length; i++)
    {
        varste.Adauga(nume[i], 15 + i);
        Console.WriteLine($"{nume[i]}: Count={varste.Count} " +
                          $"Galeti={varste.NumarGaleti}");
    }
    varste["Ion"] = 30;
    Console.WriteLine(varste["Ion"]);
    Console.WriteLine(varste.ContineCheia("Dan") + " " +
                      varste.ContineCheia("dan"));
    Console.WriteLine(varste.Sterge("Dan") + " " + varste.Sterge("Dan"));
```

```
    if (varste.IncearcaSaIa("Maria", out int v))
    {
        Console.WriteLine("Maria " + v);
    }
    bool gasit = varste.IncearcaSaIa("Dan", out int w);
    Console.WriteLine(gasit + " " + w);
    Console.WriteLine(Sortat(varste.Chei));
    Console.WriteLine(Sortat(varste.Valori));
}
```

```
Ana: Count=1 Galeti=8
Ion: Count=2 Galeti=8
Maria: Count=3 Galeti=8
Dan: Count=4 Galeti=8
Ioana: Count=5 Galeti=8
Vlad: Count=6 Galeti=8
Elena: Count=7 Galeti=16
30
True False
True False
Maria 17
False 0
Ana, Elena, Ioana, Ion, Maria, Vlad
15, 17, 19, 20, 21, 30
```

Indiciu: la redimensionare nu copiezi tabloul ca la listă. Găleata depinde de `galeti.Length`, deci fiecare cheie își calculează din nou locul.

```
public interface IMultime<T> : IColectie<T>
{
    IMultime<T> Reuniune(IColectie<T> alta);
    IMultime<T> Intersectie(IColectie<T> alta);
}
```

`MultimeHash<T> : ColectieAbstracta<T>, IMultime<T> where T : notnull`:

1. singurul câmp: `private readonly DictionarHash<T, bool> dictionar`. Cheile lui sunt elementele mulțimii. Nu moștenești dicționarul, îl **folosești**;
2. `Adauga` pe un element existent nu schimbă nimic; `Contine` e suprascris;
3. `Reuniune` / `Intersectie` întorc o mulțime **nouă**; cele două rămân neschimbate. `alta` poate fi orice colecție, chiar și o listă.

```
static void Pas5b()
{
    MultimeHash<int> pare = new MultimeHash<int>();
    MultimeHash<int> multipliDe3 = new MultimeHash<int>();
    for (int i = 0; i <= 12; i++)
    {
        if (i % 2 == 0)
        {
            pare.Adauga(i);
        }
        if (i % 3 == 0)
        {
            multipliDe3.Adauga(i);
        }
    }
    pare.Adauga(4);
    Console.WriteLine(pare.Count + " " + multipliDe3.Count);
    Console.WriteLine(Sortat(pare.Reuniune(multipliDe3)));
    Console.WriteLine(Sortat(pare.Intersectie(multipliDe3)));
```

```
    ListaTablou<int> lista = new ListaTablou<int>();
    lista.Adauga(4);
    lista.Adauga(5);
    lista.Adauga(4);
    lista.Adauga(12);
    Console.WriteLine(Sortat(pare.Intersectie(lista)));
    IMultime<int> dinLista = new MultimeHash<int>().Reuniune(lista);
    Console.WriteLine(dinLista.Count + " " + dinLista.Contine(5));
}
```

```
7 5
0, 2, 3, 4, 6, 8, 9, 10, 12
0, 6, 12
4, 12
3 True
```

Cheia ta. Clasa `Pozitie` (în `Pozitie.cs`, la rădăcina proiectului) cu `int Rand`, `int Coloana` (doar `get`) și constructorul `Pozitie(int rand, int coloana)`. Atât. Rulezi `Pas5c`. Apoi adaugi în `Pozitie` `Equals` și `GetHashCode`: două poziții sunt egale dacă au același rând și aceeași coloană. Rulezi din nou.

```
static void Pas5c()
{
    DictionarHash<Pozitie, string> tabla =
        new DictionarHash<Pozitie, string>();
    Pozitie rege = new Pozitie(1, 5);
    tabla[rege] = "rege";
    tabla[new Pozitie(8, 5)] = "regina";
    tabla[new Pozitie(8, 5)] = "tura";
    Console.WriteLine(tabla.Count);
    Console.WriteLine(tabla.ContineCheia(rege));
    Console.WriteLine(tabla.ContineCheia(new Pozitie(1, 5)));
    string? piesa;
    Console.WriteLine(tabla.IncearcaSaIa(new Pozitie(8, 5), out piesa)
        ? piesa : "pierdut");
}
```

Fără `Equals` / `GetHashCode`	Cu `Equals` / `GetHashCode`
3 True False pierdut	2 True True tura

Teorie: §5 · în `RASPUNSURI.txt`: de ce `3` și `pierdut` la prima rulare? Ce scrie compilatorul dacă suprascrii doar `Equals`?

Gata când: cele trei ieșiri sunt identice și `MultimeHash` nu are nicio buclă care să caute un element.

Pas 6 — Excepțiile ca în .NET și fail-fast

Situația	Excepția și mesajul
`this[i]`, `StergeLa(i)` cu `i` în afara `[0, Count)`; `Insereaza(i, x)` cu `i` în afara `[0, Count]`	`ArgumentOutOfRangeException`, cu `nameof(index)` și „Index was out of range. Must be non-negative and less than the size of the collection.”
`Scoate()` / `Varf()` pe stivă goală	`InvalidOperationException`: „Stack empty.”
`Iese()` / `Primul()` pe coadă goală	`InvalidOperationException`: „Queue empty.”
citire `d[k]`, cheie lipsă	`KeyNotFoundException`: „The given key 'Ion' was not present in the dictionary.”

`Adauga(k, v)`, cheie existentă	`ArgumentException`: „An item with the same key has already been added. Key: Ana”
colecția se modifică în timpul unui `foreach` pe ea	`InvalidOperationException`: „Collection was modified; enumeration operation may not execute.”

Fail-fast: `protected int versiune` în `ColectieAbstracta` (iar `DictionarHash` are propriul contor). Orice modificare reușită o crește, inclusiv `this[i] = ...`. `GetEnumerator` o reține la început și, dacă s-a schimbat, aruncă. La fel `Chei` și `Valori`.

```
static void Incearca(string eticheta, Action actiune)
{
    try
    {
        actiune();
        Console.WriteLine(eticheta + " -> ok");
    }
    catch (Exception ex)
    {
        Console.WriteLine(eticheta + " -> " + ex.GetType().Name);
    }
}

static void Pas6()
{
    ListaTablou<int> lista = new ListaTablou<int>();
    ListaInlantuita<int> inlantuita = new ListaInlantuita<int>();
    for (int i = 1; i <= 5; i++)
    {
        lista.Adauga(i);
        inlantuita.Adauga(i);
    }
    DictionarHash<string, int> d = new DictionarHash<string, int>();
    d.Adauga("Ana", 17);

    Incearca("lista[5]", () => Console.Write(lista[5]));
    Incearca("lista.Insereaza(5, 6)", () => lista.Insereaza(5, 6));
    Incearca("inlantuita.StergeLa(-1)", () => inlantuita.StergeLa(-1));
    StivaTablou<int> stiva = new StivaTablou<int>();
    CoadaCirculara<int> coada = new CoadaCirculara<int>();
    Incearca("stiva.Scoate()", () => stiva.Scoate());
    Incearca("coada.Primul()", () => coada.Primul());
    Incearca("d[\"Ion\"]", () => Console.Write(d["Ion"]));
    Incearca("d.Adauga(\"Ana\", 18)", () => d.Adauga("Ana", 18));

    try
    {
        foreach (int x in lista)
        {
            if (x % 2 == 0)
            {
                lista.Sterge(x);
            }
        }
    }
    catch (InvalidOperationException ex)
    {
        Console.WriteLine(ex.Message);
    }
    Console.WriteLine(lista);

    try
```

```
    {
        foreach (int x in inlantuita)
        {
            if (x == 5)
            {
                inlantuita.Sterge(x);
            }
        }
    }
    catch (InvalidOperationException ex)
    {
        Console.WriteLine(ex.Message);
    }
    Console.WriteLine(inlantuita);

    for (int i = lista.Count - 1; i >= 0; i--)
    {
        if (lista[i] % 2 == 1)
        {
            lista.StergeLa(i);
        }
    }
    Console.WriteLine(lista);
}
```

```
lista[5] -> ArgumentOutOfRangeException
lista.Insereaza(5, 6) -> ok
inlantuita.StergeLa(-1) -> ArgumentOutOfRangeException
stiva.Scoate() -> InvalidOperationException
coada.Primul() -> InvalidOperationException
d["Ion"] -> KeyNotFoundException
d.Adauga("Ana", 18) -> ArgumentException
Collection was modified; enumeration operation may not execute.
[1, 3, 4, 5, 6]
Collection was modified; enumeration operation may not execute.
[1, 2, 3, 4]
[4, 6]
```

Teorie: §7

Indiciu: dacă al doilea `try` nu afișează mesajul și lista rămâne `[1, 2, 3, 4]` fără excepție, verifici versiunea în locul greșit: `yield return` e locul unde `foreach`-ul se oprește și apoi revine.

Gata când: ieșirea e identică, iar `ex.Message` dă, la fiecare excepție, textul din tabel.

Pas 7 — Cât costă o căutare

```
static void Pas7()
{
    const int N = 100_000;
    ListaTablou<int> lista = new ListaTablou<int>();
    DictionarHash<int, bool> dictionar = new DictionarHash<int, bool>();
    for (int i = 0; i < N; i++)
    {
        lista.Adauga(i);
        dictionar.Adauga(i, true);
    }

    Stopwatch ceas = Stopwatch.StartNew();
    int gasite = 0;
    for (int i = 0; i < 1000; i++)
    {
```

```
        if (lista.Contine(N - 1 - i))
        {
            gasite++;
        }
    }
    ceas.Stop();
    Console.WriteLine($"ListaTablou:   {gasite} gasite in " +
                      $"{ceas.Elapsed.TotalMilliseconds:F3} ms");

    ceas.Restart();
    gasite = 0;
    for (int i = 0; i < 1000; i++)
    {
        if (dictionar.ContineCheia(N - 1 - i))
        {
            gasite++;
        }
    }
    ceas.Stop();
    Console.WriteLine($"DictionarHash: {gasite} gasite in " +
                      $"{ceas.Elapsed.TotalMilliseconds:F3} ms");
}
```

Cifrele depind de calculator. Ordinul de mărime trebuie să fie acesta:

```
ListaTablou:   1000 gasite in (sute de ms)
DictionarHash: 1000 gasite in (sutimi de ms)
```

Teorie: §8 · în `RASPUNSURI.txt`: câte comparații face, cam, fiecare variantă pentru cele 1000 de căutări?

Gata când: ambele găsesc 1000, iar dicționarul e de cel puțin 1000 de ori mai rapid.

Provocare

În academy, `Repository<T>.FindById` parcurge toată lista până găsește `Guid`-ul. Ce trebuie schimbat în `Repository<T>` ca `FindById` să caute într-un `DictionarHash<Guid, T>`, fără să parcurgă nimic? Răspunsul în `RASPUNSURI.txt`, pe codul tău de acum: fișierele, metodele și ce se schimbă în fiecare. Academy nu se modifică în etapa asta.